

课程重点

五、程序优化 ★★★★★

5.1 向量求和优化

// 原始：每次循环都调用vec_length，存在函数调用开销和边界检查

```
void vector_sum_bad(vec *v, float *sum) {  
    *sum = 0;  
    for (long int i = 0; i < vec_length(v); i++) {  
        *sum += get_vec_start(v)[i]; // 每次循环都调用get_vec_start  
    }  
}
```

// 优化：

// 1. 把vec_length移到循环外（避免每次循环调用）

// 2. 把get_vec_start移到循环外（避免重复取指针）

// 3. 用局部变量累积结果（避免每次循环读写内存）

```
void vector_sum(vec *v, float *sum) {  
    long int length = vec_length(v); // 优化1：提取到循环外  
    float *d = get_vec_start(v); // 优化2：提取到循环外  
    float t = 0; // 优化3：局部变量累积  
    for (long int i = 0; i < length; i++)  
        t += d[i];  
    *sum = t; // 最后一次性写回  
}
```

数组下标与指针等价关系：

```
int a[20];
```

```
&a[11] // 第12个元素的地址
```

```
a + 11 // 第12个元素的地址
```

```
a[11] // 第12个元素的值
```

```
*(a + 11) // 第12个元素的值
```

六、Cache计算 ★★★★★

每年必考大题，要求计算Cache结构参数、地址划分、缺失率等。

6.1 典型计算题

例1（2025A）：32KB Cache，64B/块，8路组相联，32位地址

- $B = 64 = 2^6$ ，CO = 6位

- $E = 8 = 2^3$
- $S = C/(E \times B) = 32K/(8 \times 64) = 64 = 2^6$, $CI = 6$ 位
- $CT = 32 - 6 - 6 = 20$ 位
- 索引位为A11~A6

例2 (2017A)：I7 CPU，L2 Cache为8路2M容量，B=64

- $C = 2M = 2^{21}$, $E = 8 = 2^3$, $B = 64 = 2^6$
- $S = 2^{21}/(2^3 \times 2^6) = 2^{12}$, $s = 12$

例3 (2019A)：主存地址32位，4K行，块大小16B，4路组相联

- $S = 4K/4 = 1K = 2^{10}$, $CI = 10$ 位
- $CO = \log_2 16 = 4$ 位
- $CT = 32 - 10 - 4 = 18$ 位
- 标记位总位数 = $4K \times 18 = 72K$ 位

6.3 Cache缺失率计算 ★★★★★

例 (2025A)：结合试卷查看，数组d[2048] (int型)，起始地址0x01800020，Cache 32KB/8路/64B块

- d[0]在块内偏移 = 0x20 (低6位)
- 数组占 $4 \times 2048 = 8KB$
- 每次访问读+写=2次内存访问，共4096次
- 不命中次数 = 129 (首块部分+后续每块首次访问)
- 缺失率 = $129/4096 \approx 3.15\%$
- 平均访问时间 = $2 \times 96.85\% + 200 \times 3.15\% = 8.3$ 个时钟周期

简化算法：缺失率 $\approx 1/16 = 6.25\%$ (每64B块首次访问不命中)，平均时间 = $(1/16) \times 200 + (15/16) \times 2 = 14.375$

第8章 异常、进程与信号 ★★★★★

第8章常以选择、判断、简答和综合流程题出现。核心是把异常分类、fork、execve、waitpid、信号、Shell执行流程连成一条线。

8.1 高频考点

考点	必会内容
异常分类	中断、陷阱、故障、终止的同步性和返回行为
fork	调用一次，返回两次；父子执行顺序不确定
execve	在当前进程中加载新程序，成功不返回
waitpid	回收子进程，避免僵死进程
信号	SIGCHLD、SIGINT、SIGSEGV等含义

考点	必会内容
信号处理程序	简单、只调用异步信号安全函数、保护共享数据
上下文切换	保存当前进程上下文，恢复下一个进程上下文
Shell流程	fork→execve→exit→SIGCHLD→waitpid

8.2 原题摘录：异常分类

题目：按一下回车键，在计算机系统中会导致哪类事件？

- A. 异步异常
- B. 进程切换
- C. 产生信号
- D. 陷阱（系统调用）

参考答案：A

解题要点：

- 键盘属于外部I/O设备。
- 外部设备事件与当前指令执行无关，因此属于异步异常，也就是中断。
- 系统调用才是陷阱，它是程序主动执行特殊指令进入内核。

8.3 原题摘录： fork 返回次数

题目：调用一次 fork 函数，程序会返回几次？

- A. 0次，从不返回
- B. 1次
- C. 2次
- D. 根据需要可返回任意次

参考答案：C

解释：

父进程中返回一次：返回子进程PID
子进程中返回一次：返回0

所以说： fork 调用一次，返回两次。

举一反三：

```
fork();  
fork();  
printf("X\n");
```

两次 fork 后进程数为4，所以 x 可能打印4次。若没有 waitpid 约束，打印顺序不确定。

8.4 原题摘录：子进程终止信号

题目：子进程运行终止或停止后，会向父进程发送什么信号？

参考答案： SIGCHLD

解题要点：

- 子进程终止后，内核通知父进程。
- 父进程应调用 `waitpid` 读取子进程状态并回收资源。
- Shell程序必须正确处理 `SIGCHLD`，否则会留下僵死进程。

8.5 信号处理与上下文切换速记

信号处理程序编写原则：

- 处理程序尽可能简单。
- 只调用异步信号安全的函数，例如 `write`，不要在处理程序中调用 `printf`。
- 进入处理程序后保存 `errno`，返回前恢复 `errno`。
- 访问共享全局数据结构时，阻塞相关信号，避免处理程序和主程序并发修改。
- 全局标志用 `volatile sig_atomic_t`： `volatile` 防止编译器把变量缓存到寄存器， `sig_atomic_t` 保证读写这个标志是原子的。

进程上下文：包括页全局目录 `pgd`、通用寄存器、用户栈、打开的文件表等，**不包括**内核代码和调度程序。

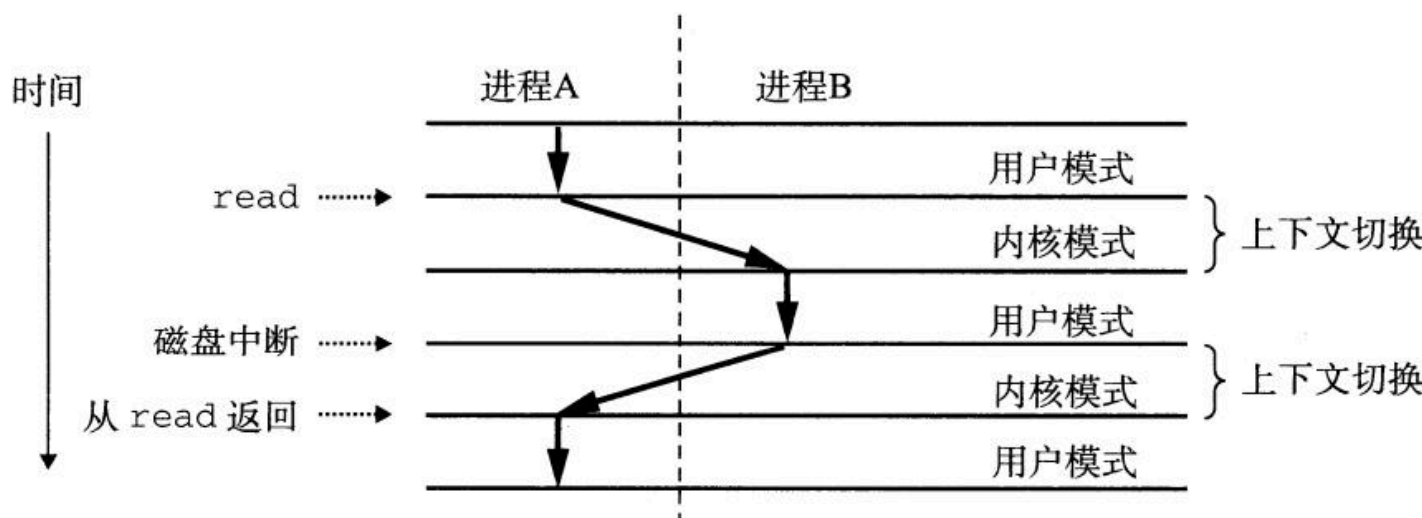


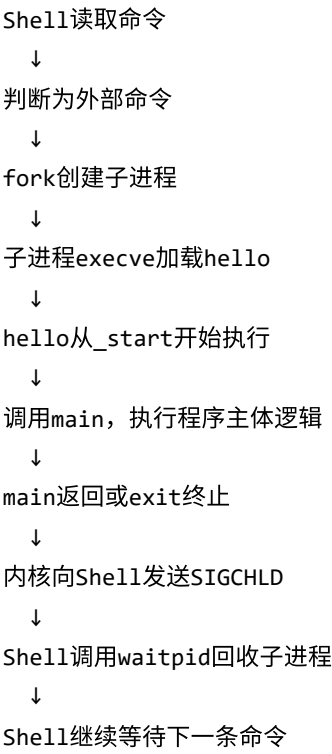
图8-14的主线是：进程A调用 `read` 陷入内核，磁盘I/O较慢，内核不空等，而是保存A的上下文、恢复B的上下文，让B运行；磁盘中断到来后，内核再根据调度情况切回A，使A从 `read` 返回后的下一条指令继续执行。

易错点：“内核代表某进程执行”不是指有一个单独的内核进程，而是CPU在该进程的上下文中以内核态运行，例如处理系统调用、保存当前进程上下文、恢复下一个进程上下文。

8.6 原题摘录：Shell执行hello全过程

题目：阐述命令行输入 `hello` 后，从进程创建到程序运行、退出、信号产生与处理的全过程。

标准答题框架：



- 答题要点：
- 必须写出 fork 和 execve 的分工。
 - 必须写出 SIGCHLD 和 waitpid 。

第9章 虚拟内存与页表 ★★★★★

第9章的考题常把概念判断和计算结合在一起。重点是页表、PTE、TLB、缺页处理、虚拟地址空间、mmap，以及与Cache地址划分的联合分析。

9.1 高频考点

考点	必会内容
虚拟内存作为缓存	物理内存缓存虚拟页
PTE	有效位、PPN、权限位、脏位、引用位
缺页	PTE无效时进入内核；地址和权限合法时调页后重新执行
TLB	页表项缓存，TLB不命中不等于缺页
多级页表	每级索引位数、节省页表空间
地址空间	.text/.data/.bss/heap/shared/stack/kernel
mmap	建立虚拟地址区域与文件/匿名对象的映射关系

核心术语表

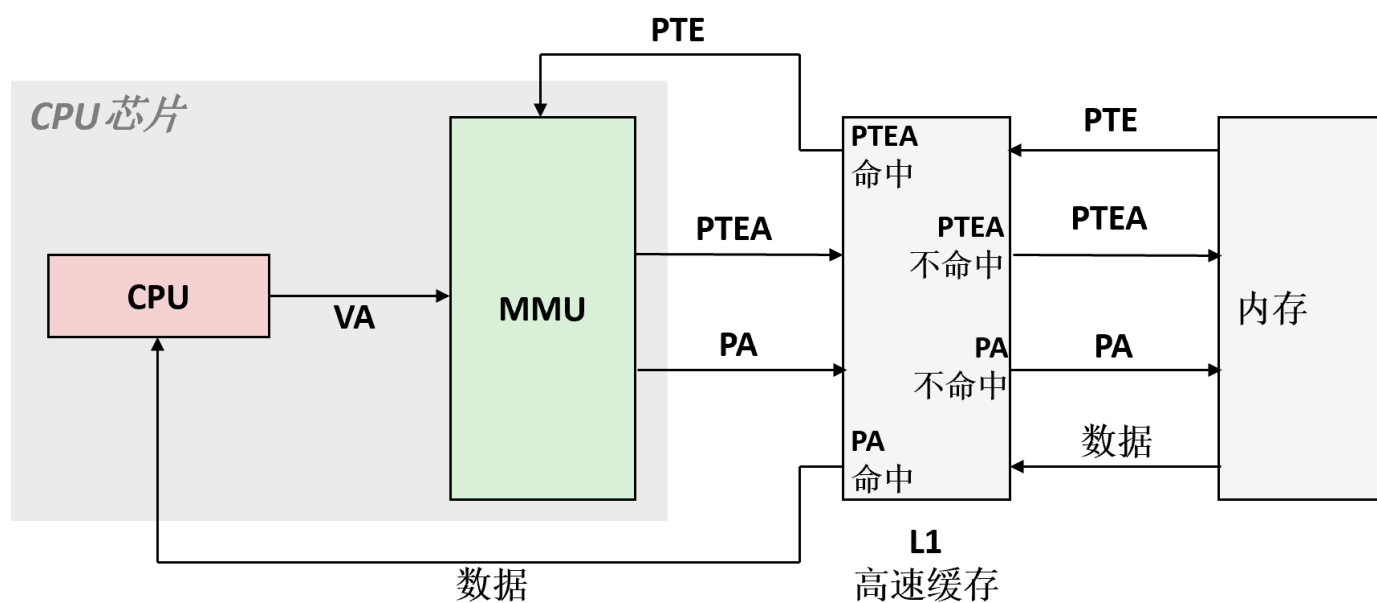
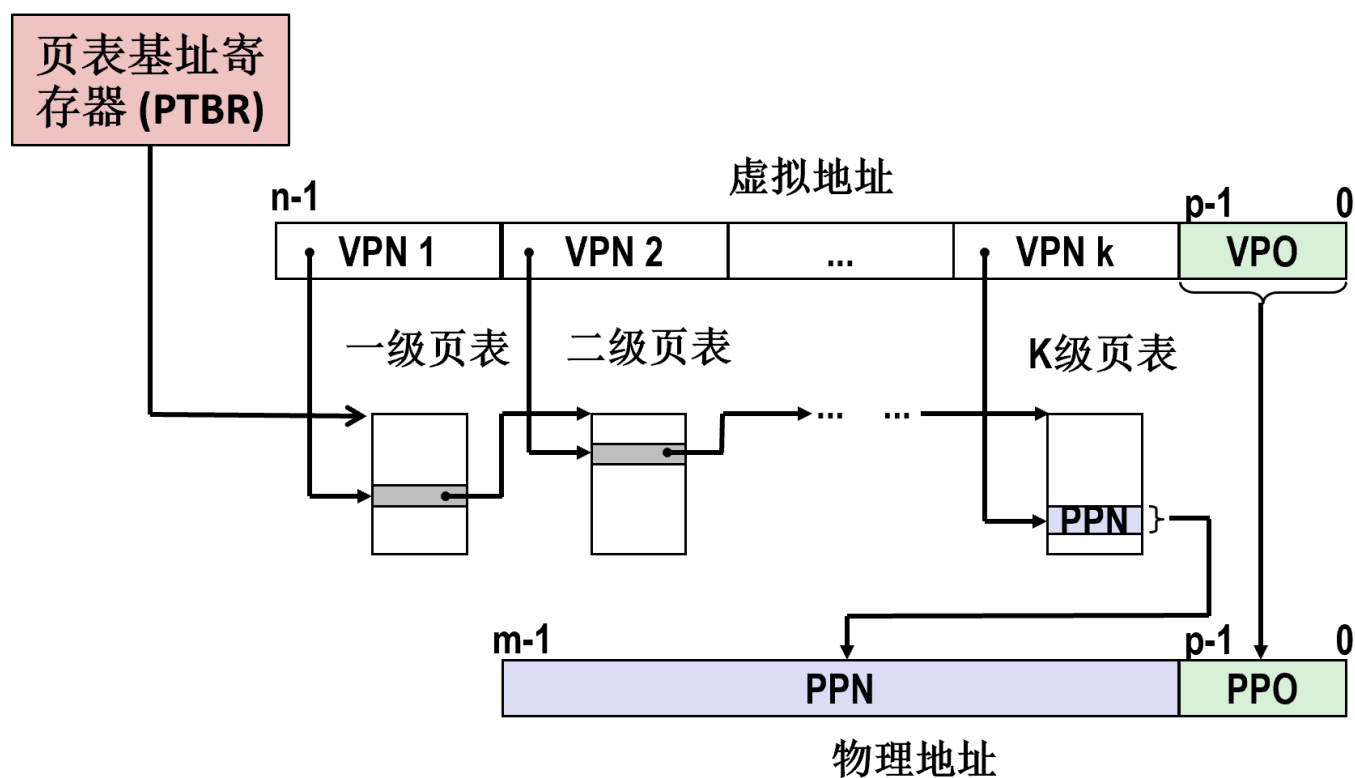
术语	含义	记忆点
VA	虚拟地址	$VA = VPN + VPO$ ，CPU发出
VPN / VPO	虚拟页号 / 页内偏移	VPN 用于查映射，VPO 原样保留
PA	物理地址	$PA = PPN + PPO$ ，Cache/主存使用
PPN / PPO	物理页号 / 物理页偏移	$VPN \rightarrow PPN$ ，且 $VPO = PPO$
页表 / PTE	映射表 / 页表项	PTE 保存有效位、权限位、PPN 等
PTEA	页表项地址	$PTEA = \text{当前页表基址} + \text{索引} \times \text{sizeof(PTE)}$
CR3	顶级页表基址寄存器	四级页表遍历从 CR3 指向的页表开始
TLB /快表	页表项缓存	缓存 $VPN \rightarrow PPN$ ，命中则不用查页表
L1 d-cache	一级数据缓存	可缓存普通数据，也可缓存 PTE 所在内存块
三类miss	TLB miss / PTEA miss / PA miss	分别表示快表无翻译、L1无页表项、L1无目标数据
缺页	page fault	PTE 表明页不能直接访问，需OS处理

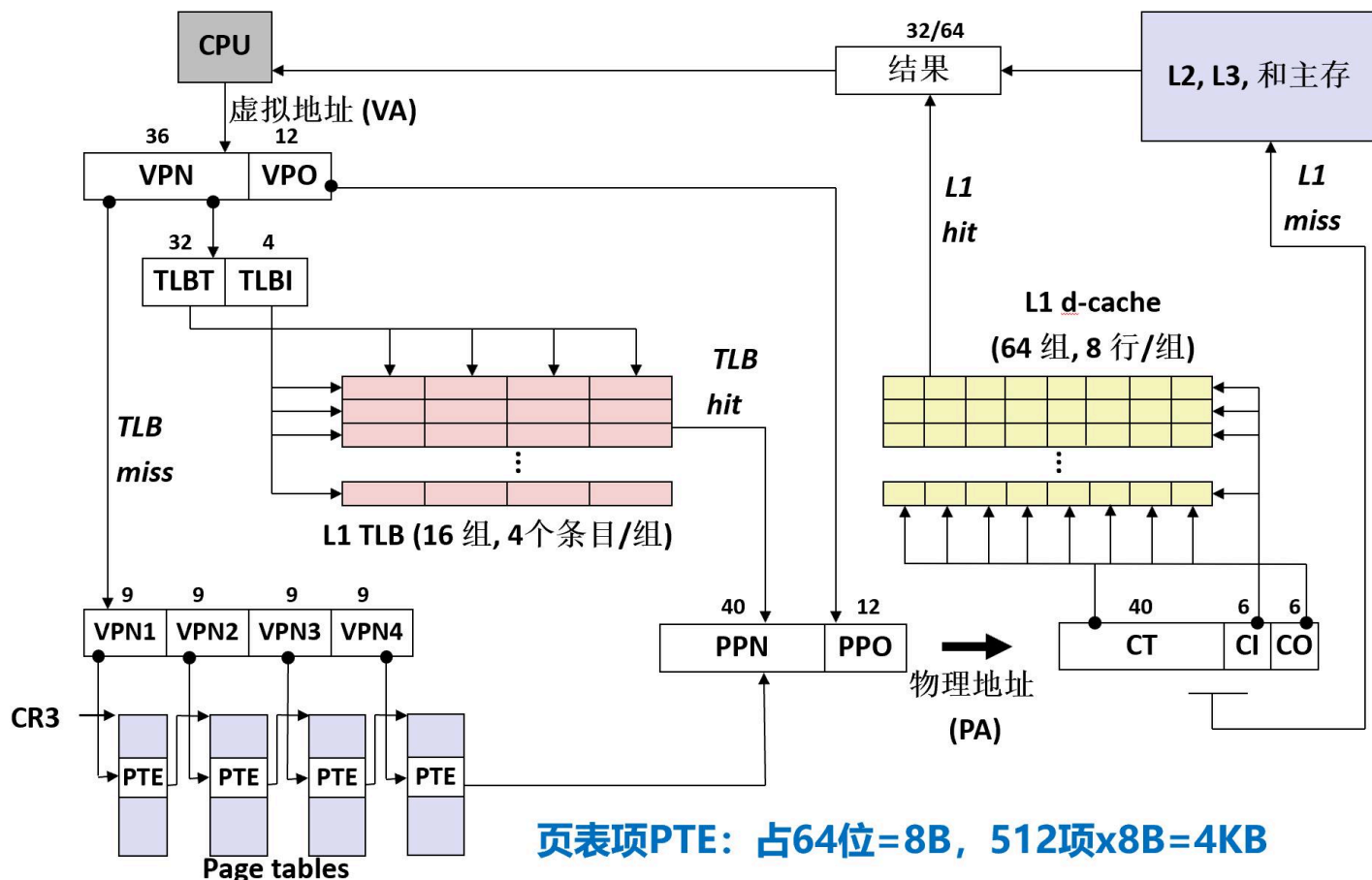
虚拟页面状态

状态	说明
未分配	尚未分配任何虚拟页或物理页
已分配未缓存	已分配虚拟页，但对应内容不在物理内存中
已分配已缓存	已分配虚拟页，且对应内容已经在物理内存中

注意：不存在“已缓存未分配”状态。

图解：页表、快表与虚拟地址到物理地址翻译





这几张图只抓一条主线：先译址，再取数；先查TLB，必要时查页表；得到PA后再访问Cache/主存。

读图要点：

- PTBR/CR3 → VPN1...VPNk → PPN：多级页表逐级索引，中间级 PTE 指向下一级页表，最后一级 PTE 给出 PPN。
- PTEA → PTE → PA → 数据：页表项 PTE 也在内存中，查页表时先用 PTEA 取回 PTE。
- VA → TLB/四级页表 → PA：TLB命中直接得 PPN；TLB不命中才从 CR3 开始逐级查页表。

先看图中的位划分：

- VA = VPN | VPO = 36位 | 12位
- VPO = 12 说明页大小是 $2^{12} = 4KB$
- 36位的 VPN 又分成 4×9 位的 VPN1~VPN4，说明这里是**四级页表**
- 每级正好9位，是因为每张页表大小为 4KB，每个 PTE 是 8B，所以一张页表有 $4096 / 8 = 512 = 2^9$ 个表项
- TLB是 16组，4项/组，所以 TLBI = 4位，TLBT = 32位
- PA = PPN | PPO = 40位 | 12位
- L1 d-cache是 64组，8行/组，64B块，所以 CI = 6位，CO = 6位，CT = 40位

页表遍历要点：

- 页表像数组，每级 PTEA = 页表基址 + 索引 × 8。
- 第一级页表基址来自 CR3；中间级 PTE 给出下一级页表基址；最后一级 PTE 给出数据页 PPN。
- TLB输入是 VPN，命中得到 PPN；L1 Cache输入是物理地址，访问 PTEA 得到 PTE，访问 PA 得到程序数据。

一次访问按图走一遍：

1. CPU给出虚拟地址 VA
2. MMU把它拆成 VPN 和 VPO

3. 查TLB；命中则直接得到 PPN
4. TLB不命中时，从 CR3 开始逐级取 PTE ，最终得到 PPN 或触发缺页
5. 用 PPN + VPO 拼出物理地址 PA
6. 用 PA 访问L1 d-cache；不命中再去L2、L3或主存

关键句：页内偏移不变，只是页号从 VPN 变成 PPN 。

易混点：

- **TLB miss**：快表里没有这条 VPN → PPN 翻译记录，不一定缺页；页可能已经在物理内存里，只是TLB还没缓存
- **PTEA miss**：用页表项地址 PTEA 查L1 Cache时，没有找到对应的页表项 PTE ；这表示L1里没有这个 PTE ，需要去L2、L3或主存取，不等于缺页
- **PA miss / L1 data cache miss**：地址翻译已经成功，但 PA 对应的数据块不在L1里，需要继续访问L2、L3或主存
- **缺页 (page fault)**：取回 PTE 后发现该页不能直接访问，需要进入内核处理
- 记忆式：TLB miss ≠ PTEA miss ≠ 缺页 ≠ PA miss

图上几个数字如何快速记忆：

- 4KB页 对应 12位页内偏移
- 512项页表 对应 9位页表索引
- 64组Cache 对应 6位组索引
- 64B块 对应 6位块内偏移
- 图上的“结果 32/64 ”表示这次读操作可能返回32位或64位数据，例如读 int 常见为32位，读 long 或指针常见为64位

考试表述抓一句即可：**CPU发出 VA ，MMU先查TLB，必要时按 CR3 和多级页表取 PTE 得到 PPN ，再与不变的页内偏移拼成 PA 访问Cache/主存。**

9.2 原题摘录：PTE中的PPN由谁写入

题目：程序加载后，页表元素PTE中的PPN物理页号是由谁写入的？

- A. fork
- B. execve
- C. 缺页异常处理子程序
- D. mmap

参考答案：C

解题要点：

- execve 建立虚拟地址区域和映射关系，但不一定立刻分配所有物理页。
- 首次访问某页时，如果页面不在物理内存，会触发缺页。
- 缺页处理程序先判断地址和权限是否合法；合法时把页面调入物理内存，并更新PTE中的PPN和有效位。
- 缺页处理主线：MMU发现PTE无效，触发缺页异常；内核必要时选择牺牲页并换出，再调入所需页，最后返回原进程重新执行故障指令。

9.3 原题摘录：页表项个数

题目：Intel Core i7每张页表中的元素个数是多少？

- A. 512
- B. 1024
- C. 2048
- D. 4096

参考答案：A

推导：

页表大小 = 4KB = 4096B

每个PTE = 8B

PTE个数 = $4096 / 8 = 512 = 2^9$

所以每级页表索引通常是9位。

9.4 原题摘录：虚拟内存与mmap判断

题目：判断正误：虚拟内存（磁盘文件）与物理内存间采用 mmap 直接映射Cache方式。

参考答案：错误

解释：

- mmap 用于建立虚拟地址区域与文件或匿名对象之间的映射，让程序像访问内存一样访问文件或匿名空间。
- 它更像是“先登记关系”，不是“立刻全部搬进物理内存”。
- 虚拟地址到物理地址的翻译依靠页表和MMU。
- 物理内存作为虚拟内存的缓存，但不是由 mmap 直接完成地址翻译。

9.5 原题摘录：execve 实现过程

题目：结合进程、信号处理、虚拟内存映射 mmap、参数表、环境变量表、动态链接等知识，阐述 execve 一直到运行 _start 的过程。

答题框架：

父进程fork创建子进程



子进程调用execve



删除旧的用户虚拟地址区域



建立新程序的虚拟内存区域



代码和初始化数据映射到目标可执行文件



.bss、栈等区域建立匿名映射



动态链接程序映射共享库和动态链接器



在用户栈中建立argv、envp



跳转到_start开始执行

答题要点：

- execve 成功后不返回原程序。
- 它不创建新进程，而是在当前进程中替换用户空间。
- 程序运行过程中可能通过缺页异常按需加载代码和数据页。